TP GraphQL

« Mes commandes » en une seule requête

IWA · DaMS5
Durée : 1 h

Vous partez d'un serveur Apollo déjà prêt à démarrer. Vous écrivez le schéma, puis les resolvers, vous voyez le problème N+1 apparaître dans les logs et vous le corrigez avec DataLoader. Quatre étapes, trois fichiers à modifier, et des tests qui vous disent quand c'est bon.

OBJECTIFS

— Écrire un schéma GraphQL (SDL) : types, `enum`, `input`, `!` et listes.
— Coder des resolvers et comprendre leurs trois arguments `(parent, args, ctx)`.
— **Voir** le problème N+1 dans les logs, puis le corriger avec DataLoader.
— Utiliser des fragments, des variables et la directive `@include`.

LIEN AVEC LE COURS — le cours de 30 minutes

`cours-graphql-dams5.pptx` : SDL (diapos 25–26), requêtes et variables (28–29), mutations (30–31), resolvers (33–34), N+1 et DataLoader (35–38), erreurs (39). Le TP suit exactement la diapo 44 « Place au TP ».

Le déroulé

	Étape	Durée	Fichier à modifier
0	Installation et prise en main	10 min	aucun
1	Écrire le schéma	10 min	`src/schema.graphql`
2	Coder les resolvers	15 min	`src/resolvers.js`
3	Repérer le N+1, le corriger	15 min	`src/loaders.js` + `src/resolvers.js`
4	Fragments et directives	10 min	`queries/etape4.graphql`
+	Bonus : mutation, erreurs	si le temps	`src/schema.graphql` + `src/resolvers.js`

La règle du jeu. Chaque étape se vérifie avec une commande `npm run test:N` (1, 2, 3 ou 4). Quand elle est verte, vous passez à la suite. Si vous êtes bloqué plus de 5 minutes, regardez le dossier `solution/` pour l'étape concernée. Ce n'est pas de la triche, mais *lisez* le corrigé au lieu de le copier : les commentaires expliquent pourquoi il est écrit comme ça.

1 Installation et prise en main 10 min

1.1 Node.js

Le serveur tourne sous Node.js **22 ou plus** (**24 LTS** recommandé). Vérifiez :

```
node -v      # doit afficher v22.x ou v24.x
npm -v
```

Si `node -v` est inconnu ou affiche une version trop ancienne, installez la version LTS :

Système	Commande
Windows	`winget install OpenJS.NodeJS.LTS` (ou l'installeur `.msi` de nodejs.org)
macOS	`brew install node@24` (ou l'installeur `.pkg` de nodejs.org)
Linux	installez nvm (une ligne, voir son README), puis `nvm install --lts`

Ensuite, **fermez et rouvrez votre terminal** : sinon il ne trouve pas `node`.

1.2 Récupérer et lancer le projet

```
git clone https://github.com/Byxis/DaMS5-TP_GraphQL.git   # ou téléchargez le .zip et
    décompressez-le
cd DaMS5-TP_GraphQL/starter
npm install                                  # ~20 secondes, 3 dépendances
npm run dev                                  # lance le serveur (redémarre à chaque sauvegarde)
```

Le terminal doit afficher `Serveur prêt : http://localhost:4000/`. Ouvrez cette adresse dans votre navigateur : c'est **Apollo Sandbox**, l'éditeur de requêtes vu en cours. Dans l'onglet *Operation*, tapez puis exécutez (bouton bleu, ou `Ctrl` + `Entrée`) :

```
query Catalogue {
  products { name price }
}
```

À droite, la Sandbox affiche les 5 produits. Regardez maintenant **le terminal où tourne `npm run dev`** : le serveur y écrit un journal à chaque requête.

```
▶ Catalogue                          ← une requête GraphQL arrive, avec son nom
  [DB #1] SELECT * FROM products     ← chaque requête faite à la base, numérotée
◀ 1 requête en base                  ← le total pour cette requête GraphQL
```

Gardez ce terminal visible pendant tout le TP : à l'étape 3, c'est lui qui vous montrera le problème N+1. Une requête sans nom, comme `{ products { name } }`, s'affiche `(requête sans nom)`.

PIÈGE — les 5 problèmes qu'on voit à chaque fois

- `'node' n'est pas reconnu...` : vous n'avez pas rouvert le terminal après l'installation.
- `npm.ps1 ne peut pas être chargé, l'exécution de scripts est désactivée` (PowerShell) : tapez `npm.cmd` au lieu de `npm`, ou une fois pour toutes `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.
- `EADDRINUSE: address already in use :::4000` : un autre serveur occupe le port. PowerShell : `$env:PORT=4001; npm run dev`. Bash : `PORT=4001 npm run dev`.
- `npm warn EBADENGINE` pendant `npm install` : un simple avertissement, ignorez-le.
- La page reste blanche : la Sandbox se charge depuis Internet. Vérifiez le Wi-Fi ou le proxy.

1.3 Le tour du projet (2 minutes)

```
starter/
  src/
    db.js            fausse base en mémoire, affiche chaque requête "SQL"    (ne pas toucher)
    server.js        assemble schéma + resolvers + contexte                   (ne pas toucher)
    index.js         démarre le serveur HTTP                                  (ne pas toucher)
    schema.js        charge schema.graphql                                    (ne pas toucher)
    schema.graphql   le CONTRAT : types GraphQL                               → étape 1
    resolvers.js     QUI VA CHERCHER QUOI                                     → étapes 2 et 3
    loaders.js       le DataLoader                                            → étape 3
  queries/etape4.graphql                                                      → étape 4
  tests/             les tests qui vous corrigent                             (ne pas toucher)
```

Lancez les tests dans un **deuxième terminal** (laissez le serveur tourner dans le premier) :

```
npm test           # tout : au début, c'est rouge partout, et c'est normal
npm run test:1     # seulement l'étape 1 (puis test:2, test:3, test:4, test:bonus)
```

POINT D'ÉTAPE

La Sandbox affiche les 5 produits, le terminal affiche `[DB #1] SELECT * FROM products`, et `npm test` s'exécute (en rouge, c'est normal).

2 Étape 1 — Le schéma 10 min

On reprend la boutique du cours : Inès (id `42`) a trois commandes, et l'écran « Mes commandes » veut afficher, pour chacune, son statut et le nom du produit.

RAPPEL — la SDL en 30 secondes

`Int Float String Boolean ID`	les 5 scalaires
`String!`	`!` = jamais `null`
`[Order!]!`	une liste non nulle, d'éléments non nuls
`enum E { A B C }`	un ensemble fini de valeurs
`input X { ... }`	un type *d'entrée*, pour les arguments
`type Query { ... }`	le point d'entrée : tout part d'ici

Le schéma s'écrit en SDL pur, dans `src/schema.graphql` (comme dans les diapos). Le type `Product` vous y est donné en exemple :

```
"Un article vendu par la boutique."
type Product {
  id: ID!
  name: String!
  price: Float!
}
```

À VOUS 1 — compléter src/schema.graphql ★☆☆

Ajoutez dans `src/schema.graphql`, dans la zone marquée `ÉTAPE 1` et dans `type Query` :

1. `OrderStatus`, le statut d'une commande : en attente (`PENDING`), expédiée (`SHIPPED`) ou livrée (`DELIVERED`), et rien d'autre ;

2. le type `Order`, avec des champs tous non nuls : `id`, `status` représentant le statut de la commande, `quantity` en entier, un flottant `total` et un produit `product` ;
3. le type `User`, avec des champs tous non nuls : `id`, un `name` et ses commandes `orders`. La liste existe toujours (vide au pire) et ne contient jamais de `null` ;
4. `CreateOrderInput`, les données envoyées pour passer une commande : l'identifiant du produit `productId` et une `quantity` entière, toutes deux obligatoires. Attention, ce n'est pas un `type` comme les autres. Il servira au bonus ;
5. dans `Query`, deux points d'entrée : `user`, qui prend un `id` obligatoire et renvoie l'utilisateur correspondant, ou rien s'il n'existe pas ; et `orders`, qui renvoie toutes les commandes (mêmes règles que `User.orders`).

Les tests vous indiquent le type attendu pour chaque champ si vous vous trompez.
Vérifiez : `npm run test:1` → 6 tests verts.

Si le terminal du serveur affiche une erreur rouge (par exemple `Unknown type "Order"`), votre schéma ne compile pas : corrigez-le et sauvegardez, le serveur redémarre tout seul.

QUESTION FLASH Q1 — `user(id)` : avec ou sans ! ?

Que doit renvoyer le serveur si on demande `user(id: "999")` et que cet utilisateur n'existe pas ? Qu'est-ce que ça impose sur le type de retour de `user` ?

PIÈGE — il y a un email en base, mais pas dans le schéma

Dans `db.js`, chaque utilisateur a un `email`. Ne l'ajoutez **pas** au schéma (un test le vérifie) : aucun écran n'en a besoin, et c'est une donnée personnelle. Le schéma est un *contrat* : le client ne peut demander que ce qui y figure. Essayez `{ user(id: "42") { email } }` dans la Sandbox : refus immédiat, avant même d'exécuter quoi que ce soit.

Retournez dans la Sandbox et ouvrez l'onglet *Schema* (icônes à gauche) : vos nouveaux types y sont déjà. La Sandbox les a obtenus en interrogeant le serveur lui-même : c'est l'**introspection**. Maintenant, lancez :

```
query Commandes { orders { id status } }
```

Vous obtenez une erreur : `Cannot return null for non-nullable field Query.orders` (lisez seulement le champ `message` ; la `stacktrace` sert à déboguer côté serveur). **C'est normal, votre schéma est juste** : il *promet* une liste, mais aucun code ne sait encore où aller la chercher. C'est l'objet de l'étape 2.

POINT D'ÉTAPE

`npm run test:1` est vert, et la Sandbox affiche vos types dans l'onglet *Schema*.

3 Étape 2 — Les resolvers 15 min

RAPPEL — ce qu'est un resolver

Une fonction qui dit à GraphQL **comment obtenir la valeur d'un champ**. Elle reçoit toujours les trois mêmes arguments :

— `parent` : l'objet du niveau au-dessus (pour `Order.product`, c'est la commande) ;

— `args` : les arguments du champ (pour `user(id: "42")`, c'est `{ id: "42" }`) ;

— `ctx` : le contexte de la requête : `ctx.db`, `ctx.loaders`, `ctx.currentUserId`.

Pas besoin de resolver pour un champ qui existe déjà tel quel dans l'objet parent (`name`, `status`, `total`...) : GraphQL le lit tout seul.

Toute la difficulté est là : la base stocke des **clés étrangères**, le schéma expose des **objets**.



Les fonctions de la base dont vous avez besoin, et d'où vient la valeur à leur passer :

Champ du schéma	Fonction de `db.js`	La valeur vient de
`Query.user`	`ctx.db.userById(...)`	`args.id`
`Query.orders`	`ctx.db.allOrders()`	(rien à passer)
`User.orders`	`ctx.db.ordersByUserId(...)`	`user.id` (le parent)
`Order.product`	`ctx.db.productById(...)`	`order.productId` (le parent)

Ces fonctions renvoient des promesses : pas besoin de `async/await`, renvoyez directement la promesse, GraphQL attend le résultat tout seul.

À VOUS 2 — compléter src/resolvers.js ★☆☆

Sous chaque commentaire `ÉTAPE 2`, retirez les `//` devant le code, puis remplacez chaque `....` Inspirez-vous de `Query.products`, déjà écrit. Le résultat doit avoir cette forme :

```
export const resolvers = {
  Query: {
    products: (_parent, _args, ctx) => ctx.db.allProducts(),   // donné
    user:     (_parent, args, ctx)  => ...,   // indice : l'id est dans args
    orders:   (_parent, _args, ctx) => ...,
  },
  User: {
    orders:  (user, _args, ctx)  => ...,      // indice : user est le PARENT
  },
  Order: {
    product: (order, _args, ctx) => ...,      // indice : order.productId
  },
};
```

Vérifiez : `npm run test:2` → 5 tests verts.

Testez maintenant la requête de la diapo 29, avec une **variable**. Dans la Sandbox, collez l'opération, puis `{ "id": "42" }` dans le panneau *Variables* en bas :

```
query CommandesClient($id: ID!) {
  user(id: $id) {
    name
    orders {
      status
      product { name price }
    }
  }
}
```

PIÈGE — n'écrivez jamais `user(id: "' + id + '")`

Une variable est envoyée *à part* et validée par le schéma (`$id: ID!`). Concaténer une chaîne dans la requête, c'est l'équivalent GraphQL de l'injection SQL, et ça casse le cache des requêtes. Même logique que les requêtes préparées en SQL.

POINT D'ÉTAPE

`npm run test:2` est vert, et `CommandesClient` affiche les 3 commandes d'Inès.

4 Étape 3 — Le problème N+1 15 min

4.1 Le constater

Dans la Sandbox, exécutez cette requête et **regardez le terminal du serveur** :

```
query MesCommandes {
  orders { status product { name } }
}
```

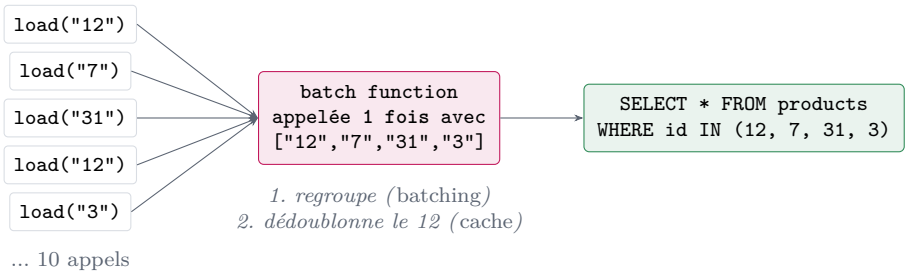
```
▶ MesCommandes
  [DB #1] SELECT * FROM orders
  [DB #2] SELECT * FROM products WHERE id = 12
  [DB #3] SELECT * FROM products WHERE id = 7
  [DB #4] SELECT * FROM products WHERE id = 31
  [DB #5] SELECT * FROM products WHERE id = 12      ← encore le 12 !
  ...
  [DB #11] SELECT * FROM products WHERE id = 31
◀ 11 requêtes en base
```

1 requête pour la liste, puis **une par commande** pour son produit : c'est le **N+1**. Le resolver `Order.product` est appelé 10 fois, une fois par commande, et chaque appel fait sa propre requête. Avec 30 ms par requête, on attend déjà 330 ms. Avec 1 000 commandes, il faut 1 001 requêtes.

QUESTION FLASH Q2 — compter les requêtes

Sans DataLoader, combien de requêtes en base pour `{ user(id: "42") { name orders { product { name } } } }` ? (Inès a 3 commandes.) Vérifiez ensuite dans le terminal.

4.2 Le corriger avec DataLoader



Au lieu d'interroger la base tout de suite, chaque resolver appelle `loader.load(id)`. DataLoader attend que les 10 resolvers `Order.product` aient tous fait leur `.load()` (c'est quasi instantané), puis appelle **une seule fois** votre *batch function* avec la liste des ids, sans doublons. Cette fonction a un contrat strict :

RAPPEL — le contrat d'une batch function

Elle reçoit un tableau d'ids et renvoie (une promesse de) un tableau de résultats : **de même longueur**, **dans le même ordre** que les ids, avec `null` pour un id introuvable.

Le piège, c'est l'ordre : la base renvoie les produits dans *son* ordre, pas dans celui des ids. Il faut donc les remettre en place. Un exemple :

```
ids      = ["31", "7", "999"]                    ← ce que DataLoader vous demande
rows     = [ { id: "7",  name: "Clavier" },     ← ce que renvoie la base
             { id: "31", name: "Souris"  } ]       (son ordre à elle, et pas de 999)
résultat = [ Souris, Clavier, null ]             ← ce que vous devez renvoyer
```

Pour chaque id demandé, on cherche *son* produit dans `rows`. Deux outils JavaScript suffisent :

- `ids.map((id) => ...)` construit un nouveau tableau, un élément par id, dans le même ordre ;
- `rows.find((product) => condition)` renvoie le premier produit pour lequel la condition est vraie, ou `undefined` s'il n'y en a aucun ; `?? null` transforme ce `undefined` en `null`.

À VOUS 3 — brancher DataLoader ★★☆

1. Dans `src/loaders.js`, décommentez le squelette (retirez les `//`) et supprimez la ligne `product: null`. Il ne reste qu'**une condition** à écrire :

```
product: new DataLoader(async (ids) => {
  // 1. UNE seule requête pour tous les ids : WHERE id IN (...)
  const rows = await db.productsByIds(ids);

  // 2. Pour CHAQUE id demandé, chercher SON produit dans rows.
  return ids.map((id) => rows.find((product) => /* À VOUS : la condition */) ?? null);
}),
```

2. Dans `src/resolvers.js`, faites passer `Order.product` par le loader : son corps devient `ctx.loaders.product.load(order.productId)`.
3. Relancez `MesCommandes` dans la Sandbox.
Vérifiez : `npm run test:3` → 3 tests verts, et dans le terminal :

```
▶ MesCommandes
  [DB #1] SELECT * FROM orders
  [DB #2] SELECT * FROM products WHERE id IN (12, 7, 31, 3)
◀ 2 requêtes en base
```

PIÈGE — un DataLoader par requête, jamais un seul pour tout le serveur

Ouvrez `src/server.js` : `createContext()` crée de **nouveaux** loaders à chaque requête HTTP. Avec un loader global, son cache serait partagé entre tous les utilisateurs et ne se viderait jamais : on servirait des prix périmés, voire les données d'un autre client.

EN ENTREPRISE — pourquoi c'est vital dans une gateway

Dans l'architecture du cours (diapo 42), `productById` n'est pas une requête SQL mais un `GET /products/12` vers un autre microservice. Le N+1 devient alors 11 appels réseau, et leurs latences s'additionnent. Toute gateway GraphQL sérieuse utilise DataLoader ou un équivalent.

POINT D'ÉTAPE

`npm run test:3` est vert, et le terminal affiche **2 requêtes** au lieu de 11.

5 Étape 4 — Fragments et directives 10 min

Côté client uniquement, sans code serveur : tout se passe dans la Sandbox.
Une **opération**, c'est ce que vous envoyez depuis la Sandbox : `query Nom($variables) { ... }`, comme `Catalogue` ou `CommandesClient`. Ici, vous allez **améliorer** une opération déjà écrite, `MonProfil`, avec deux outils :

Outil	À quoi ça sert	Où ça s'écrit
Directive	décider si un champ est demandé ou non	sur un champ : `orders @include(if: $details)`
Fragment	ne pas répéter la même liste de champs	défini sous l'opération, utilisé avec `...Nom`

La syntaxe (ce ne sont pas des requêtes complètes) :

```
# Un FRAGMENT : un morceau de sélection réutilisable, sur un type donné.
fragment InfosProduit on Product { name price }
#  ... et on l'utilise avec ... :   product { ...InfosProduit }

# Une DIRECTIVE : modifie l'exécution. Le champ n'est demandé que si $details vaut true.
orders @include(if: $details) { status }        # @skip(if: ...) fait l'inverse
```

À VOUS 4 — améliorer MonProfil ★★☆

Ouvrez `queries/etape4.graphql` et collez son contenu dans la Sandbox, avec les variables `{ "id": "42", "details": true }`. La requête est déjà écrite, mais elle a deux défauts.

1. La directive. La Sandbox répond `Variable "$details" is never used` : la variable est déclarée, mais rien ne s'en sert. Ajoutez `@include(if: $details)` sur le champ `orders`. Testez avec `"details": true`, puis `false` : les commandes doivent apparaître, puis disparaître.

2. Le fragment. `name` et `price` sont écrits deux fois (produits des commandes, et catalogue). Sous la requête, définissez un fragment `InfosProduit` sur `Product` qui contient ces deux champs, puis remplacez les deux sélections par `...InfosProduit`. La réponse ne doit pas changer.

3. Recopiez la requête complète, fragment compris, dans `queries/etape4.graphql` : c'est ce fichier que lit le test.
Vérifiez : `npm run test:4` → 4 tests verts.

QUESTION FLASH Q3 — à quoi sert vraiment un fragment ?

On veut aussi afficher l'`id` de chaque produit. Combien d'endroits faut-il modifier dans `MonProfil` avec le fragment ? Et sans ?

POINT D'ÉTAPE — fin du TP

`npm test` affiche **18 tests verts** (et 7 `skipped` : ceux du bonus). Bravo, vous avez une API GraphQL complète, sans N+1.

6 Bonus — pour ceux qui ont fini en avance

Les tests du bonus sont ignorés (`skipped`) tant que vous n'avez rien écrit, puis s'activent tout seuls. Vérifiez avec `npm run test:bonus`.

À VOUS 5 — la mutation createOrder (diapos 30–31) ★★☆

Ajoutez au schéma, en suivant la convention *input / payload* du cours :

```
type Mutation {
  createOrder(input: CreateOrderInput!): CreateOrderPayload!
}
type CreateOrderPayload {
  order: Order                 # null si la commande est refusée
  userErrors: [UserError!]!    # vide si tout va bien
}
type UserError {
  message: String!
  field: String
}
```

Puis ajoutez un bloc `Mutation` dans `src/resolvers.js`, avec le resolver `createOrder: async (_parent, { input }, ctx) => { ... }`. Pour lire le produit : `await ctx.loaders.product.load(input.productId)`. Les règles :

Cas	`message`	`field`
`quantity <= 0`	libre	`"quantity"`
produit introuvable	libre	`"productId"`
`product.stock < quantity`	`"Stock épuisé"`	`"quantity"`
sinon → `ctx.db.insertOrder({ userId, productId, quantity })`, avec `userId = ctx.currentUserId`		

En cas de refus : `{ order: null, userErrors: [ ... ] }`. En cas de succès : `{ order, userErrors: [] }`. La Webcam (id 18) est en rupture de stock, pour tester.

PIÈGE — qui passe la commande ?

L'utilisateur vient de `ctx.currentUserId` (l'authentification), **jamais** de l'input : sinon n'importe qui pourrait commander au nom de n'importe qui. C'est pour ça que `CreateOrderInput` n'a pas de champ `userId`.

À VOUS 6 — data et errors dans la même réponse (diapo 39) ★☆☆

Ajoutez `order(id: ID!): Order` à `Query`. Si la commande n'existe pas, le resolver **lève** une erreur au lieu de renvoyer `null` :

```
import { GraphQLError } from 'graphql';
// ...
throw new GraphQLError('Commande introuvable', { extensions: { code: 'NOT_FOUND' } });
```

Demandez `{ order(id: "999") { id } }` : la réponse contient `data.order = null` **et** un tableau `errors` avec le `path`. Le statut HTTP vaut pourtant **200** (affiché au-dessus de la réponse dans la Sandbox). Comparez avec `{ user(id: "42") { email } }` qui renvoie **400** : la requête est refusée avant même d'être exécutée.

À VOUS 7 — l'introspection (diapo 27) ★☆☆

Exécutez `{ __type(name: "Order") { fields { name description } } }`. D'où vient l'autocomplétion de la Sandbox ? Pourquoi la désactive-t-on souvent en production ?

7 À retenir

Question	Réponse
Qui décrit les données ?	Le **schéma** : le contrat, typé.
Qui va les chercher ?	Les **resolvers** `(parent, args, ctx)`.
Quel piège ?	Le **N+1** : un resolver de liste × N éléments.
Comment le limiter ?	**DataLoader** : batching + cache, un par requête.
Et si une partie échoue ?	`data` **et** `errors` coexistent, même en HTTP 200.
GraphQL, REST ou gRPC ?	Selon le besoin : GraphQL côté clients, REST/gRPC derrière.